



Parse(), TryParse() и Convert в C#

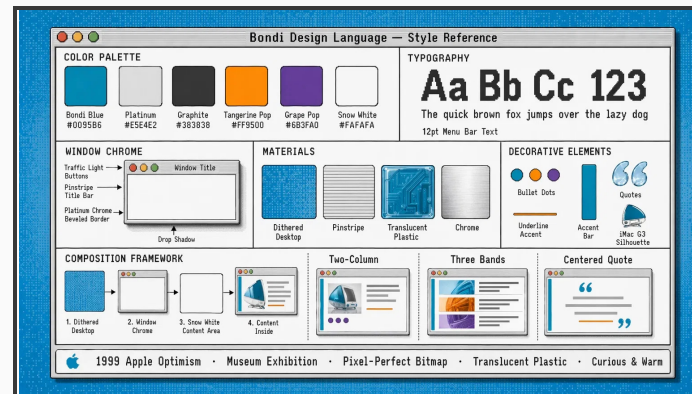
Как безопасно и правильно преобразовывать
данные

Плотная памятка по выбору метода: строгий парсинг, безопасная
проверка и универсальное преобразование базовых типов.

Parse()

TryParse()

Convert



Шаблон: Bondi Blue desktop, chrome window,
pixel-perfect labels, translucent iMac G3
accents.



Три способа преобразования данных

Parse()	TryParse()	Convert
Главная идея Преобразует строку в нужный тип данных.	Главная идея Пробует преобразовать строку без падения программы.	Главная идея Универсально переводит базовые типы друг в друга.
Когда нужен Когда строка заранее известна как корректная.	Когда нужен Когда данные могут быть ошибочными или внешними.	Когда нужен Когда входные данные не только строки.
Риск Выбрасывает исключение при ошибке.	Риск Нужно обязательно проверить `true/false`.	Риск При неверном формате ведет себя как Parse().

KEY

Главное различие: **реакция на ошибки**, **тип входных данных** и **стабильность программы**.



Parse(): быстрый, но строгий

Назначение

Приведение строки к нужному типу данных.

Вход

Принимает исключительно строковый тип:
string.

Успех

Возвращает готовое значение нужного типа.

Ошибка

Прерывает работу программы и выбрасывает исключение.

Скорость

Очень высокая при успешном выполнении.

Цена ошибки

Исключения нагружают процессор и память.

STRICT CONVERSION FLOW

"123" → int

valid string → value

bad string → exception

Использовать только тогда, когда корректность строки гарантирована заранее.

Parse() – это инструмент для контролируемых данных, а не для случайного ввода.



Когда Parse() ломает программу

NULL
INPUT

`ArgumentNullException`

Пустое значение не может быть разобрано как корректная строка.

BAD
FORMAT

`FormatException`

Буквы в числе, лишние символы или неподходящий формат приводят к сбою.

RANGE
OVERFLOW

`OverflowException`

Слишком большое значение не помещается в диапазон целевого типа.

WARNING

Не применяйте `Parse()` к пользовательскому вводу, файлам и сетевым данным без предварительной проверки.

EXCEPTION COST



Программа прерывает обычный поток выполнения.



Создание исключений нагружает процессор.

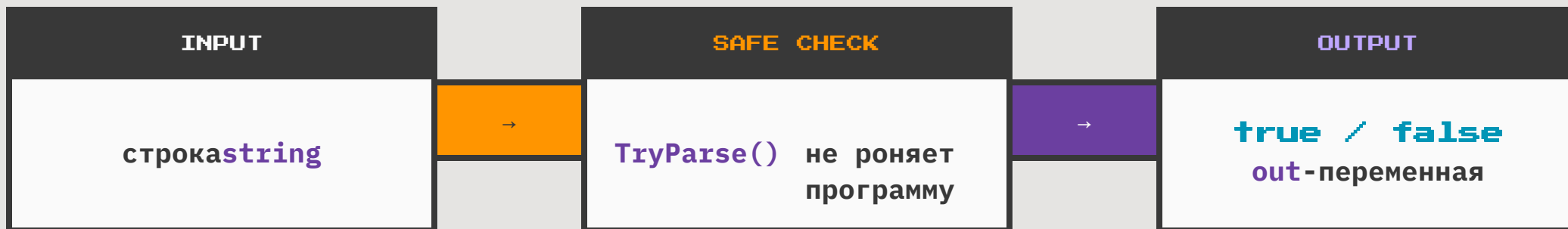


Частые ошибки увеличивают расход памяти.

Ошибка для `Parse()` – это не статус, а аварийная ситуация.



TryParse(): безопасная альтернатива

**Назначение**

Безопасная попытка преобразовать строку в нужный тип данных.

Поведение при ошибке

Метод гарантированно не выбрасывает исключений: вместо падения возвращается **false**.

Возврат операции

При успехе возвращается **true**, при любой ошибке формата, переполнении или **null** – **false**.

Куда попадает значение

Результат записывается в выходную переменную **out**; при ошибке она получает значение по умолчанию, например **0**.

RULE

Если данные ненадежные, **TryParse()** превращает ошибку в проверяемый результат, а не в исключение.



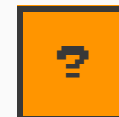
TryParse() выигрывает на ненадежных данных

Источник	Типичный риск	Почему TryParse()
Ввод пользователя	Опечатки, пустые строки, неверный формат	Возвращает false вместо падения программы
Текстовые файлы	Поврежденные, неполные или смешанные данные	Позволяет пропустить ошибочную строку
Внешняя сеть	Нельзя гарантировать формат входящих значений	Безопасно проверяет каждое значение
Массовая обработка	Ошибки встречаются часто и становятся нормой	Не создает дорогие исключения

STABLE PARSING MODE



Исключений нет: метод гарантированно не выбрасывает Exception.



Проверка явная: результат операции – true или false.



Значение отдельно: итог записывается в выходную переменную.

Если ошибка во входных данных –**обычная ситуация**, выбирайте TryParse(), а не Parse().



Convert: универсальный преобразователь

Назначение

Универсальный системный класс для преобразования **базовых типов данных** друг в друга.

Входные данные

Работает не только со строками: принимает типы, реализующие интерфейс **IConvertible**.

Примеры

Подходит для сценариев вроде **double** → **int**, **int** → **bool**, **string** → **int**.

null

Если передать пустое значение вместо строки, Convert возвращает значение по умолчанию: например, **0** для чисел.

Ошибки

При неверном формате строки или переполнении диапазона выбрасывает исключение, как **Parse()**.

TYPE ROUTER / BASIC CONVERSIONS

double**int****int****bool****string****int**

Convert удобен, когда задача шире обычного парсинга строки.

RULE

Выбирайте Convert, когда нужно переводить разные базовые типы или безопасно обработать null как значение по умолчанию.



Особенность Convert: null не опасен

Ситуация	Parse()	TryParse()	Convert
Передан null пустое значение вместо строки	ERROR Выбрасывает <code>ArgumentNullException</code> .	SAFE Возвращает <code>false</code> , а переменная получает значение по умолчанию.	SAFE Возвращает значение по умолчанию: например, <code>0</code> или <code>false</code> .
Неверный формат например, буквы в числе	ERROR Выбрасывает <code>FormatException</code> .	SAFE Возвращает <code>false</code> без исключения.	ERROR Ведет себя как <code>Parse()</code> : выбрасывает <code>FormatException</code> .
Переполнение диапазона слишком большое число	ERROR Выбрасывает <code>OverflowException</code> .	SAFE Возвращает <code>false</code> , результат становится значением по умолчанию.	ERROR Выбрасывает <code>OverflowException</code> .

KEY NOTE

Convert безопасен именно при `null`, но не является полной заменой **TryParse()**: неверный формат и переполнение всё равно приводят к исключениям.



Округление Convert может удивить

BANKER'S ROUNDING

1

При преобразовании вещественного числа в целое Convert округляет значение к ближайшему целому.

½

Если число находится ровно посередине между двумя целыми, выбирается ближайшее четное число.

Σ

Такой подход уменьшает накопленную ошибку при большом количестве вычислений.

EXAMPLES / HALF TO EVEN

Вход	Обычное ожидание	Convert
2.5	3	2 четное
3.5	4	4 четное
4.5	5	4 четное
5.6	6	6 ближайшее

WHY IT MATTERS

В финансовых, статистических и массовых расчетах важно помнить: **.5 не всегда округляется вверх**; Convert применяет правило «к ближайшему четному».



Итоговое правило выбора

Если ситуация такая	Выбирай
Данные ввел пользователь: возможны опечатки, пустые строки и неверный формат.	TryParse()
Данные пришли из текстового файла или внешней сети: формат нельзя гарантировать.	TryParse()
Строка создана и полностью контролируется вашим кодом.	Parse()
Ошибка формата должна считаться критическим системным сбоем.	Parse()
Нужно преобразовать не только строку, а разные базовые типы данных.	Convert
<code>null</code> должен превратиться в значение по умолчанию без падения программы.	Convert

MEMORY FORMULA	
Try Parse	Для ненадежных данных: пользователь, файл, сеть, массовая проверка.
Parse	Для гарантированно правильной строки, где ошибка должна быть исключением.
Convert	Для универсального перевода базовых типов и спокойной обработки <code>null</code> .
Выбор зависит от источника данных.	

RULE

Ненадежные данные → TryParse(); контролируемая строка → Parse(); разные типы или `null` по умолчанию → Convert.



Главный вывод

Метод	Сильная сторона	Главный риск
<code>Parse()</code>	Максимально быстрый на корректных строках, когда формат заранее контролируется кодом.	Падает при ошибочных данных: <code>null</code> , неверный формат и переполнение превращаются в исключения.
<code>TryParse()</code>	Самый безопасный для внешнего ввода: пользователя, файлов, сети и массовой обработки.	Требует проверки результата <code>true/false</code> ; без проверки можно потерять факт ошибки.
<code>Convert</code>	Универсален для базовых типов и устойчив к <code>null</code> , возвращая значение по умолчанию.	Не защищает от неверного формата и переполнения: возможны исключения, как у <code>Parse()</code> .

DECISION DISK

1

Данные ненадежные или внешние → `TryParse()`

2

Строка гарантированно корректна → `Parse()`

3

Преобразуются разные базовые типы → `Convert`

Выбор метода – это выбор поведения программы при ошибке.

REMEMBER

Для надежного кода выбирайте метод не по привычке, а по источнику данных и ожидаемой реакции программы на ошибку.